



API Documentation

class MedicalInterfaceBackend

doctors

nurses

patients

lab_orders

currentUserPermissions

currentUserCredentials

CreateDoctor()

CreateNurse()

CreatePatient()

CreateLabOrder()

GetLabOrdersForPatient()

GetAllLabOrders()

Login()

load_all()

save_all()

class Doctor

Doctor()

firstName

lastName

role

specialty

permissions

DisplayInfo()

```
UpdateInformation()
class Nurse
Nurse()
firstName
lastName
role
department
permissions
DisplayInfo()
class Patient
Patient()
patient_id
firstName
lastName
dateOfBirth
role
permissions
DisplayInfo()
class LabOrder
LabOrder()
order_id
patient_id
ordering_doctor_id
test_name
notes
status
Display()
ClearConsole()
prompt()
pause()
run_menu()
```

```
ViewStaffScreen()
CreateDoctorScreen()
CreateNurseScreen()
StaffManagementMenu()
CreatePatientScreen()
ViewAllPatientsScreen()
ViewPatientByIDScreen()
PatientManagementMenu()
OrderLabTestScreen()
ViewAllLabOrdersScreen()
ViewLabOrdersByPatientScreen()
ClinicalActionsMenu()
CredentialsMenu()
LoginMenu()
userMenu()
```

built with 

doctor_module

• [View Source](#)

class **MedicalInterfaceBackend:**

• [View Source](#)

Central backend for the medical interface system.

Manages all staff (doctors, nurses), patients, and lab orders. Handles login/authentication, role-based permission enforcement, and JSON-based persistence to disk.

Attributes: doctors (dict): Maps doctor ID strings to Doctor objects. nurses (dict): Maps nurse ID strings to Nurse objects. patients (dict): Maps patient ID strings to Patient objects. lab_orders (list): List of all LabOrder objects in the system. currentUserPermissions (dict): Permission flags for the currently logged-in user. currentUserCredentials (dict): Credentials/metadata for the currently logged-in user.

doctors

nurses

patients

lab_orders

currentUserPermissions

currentUserCredentials

```
def CreateDoctor(  
    self,  
    strUserIdentification,  
    strPassword,  
    strFirstName,  
    strLastName,  
    strPhoneNumber,  
    strSpecialty  
):
```

[• View Source](#)

Register a new doctor in the system.

Args: strUserIdentification (str): Unique doctor ID / username. strPassword (str): Login password. strFirstName (str): Doctor's first name. strLastName (str): Doctor's last name. strPhoneNumber (str): Contact phone number. strSpecialty (str): Medical specialty (e.g. "Pediatric").

Returns: bool: True if the doctor was created successfully, False if the ID already exists.

```
def CreateNurse(  
    self,  
    strUserIdentification,  
    strPassword,  
    strFirstName,  
    strLastName,  
    strPhoneNumber,  
    strDepartment  
):
```

[• View Source](#)

Register a new nurse in the system.

Args: strUserIdentification (str): Unique nurse ID / username. strPassword (str): Login password. strFirstName (str): Nurse's first name. strLastName (str): Nurse's last name.

strPhoneNumber (str): Contact phone number. strDepartment (str): Department the nurse belongs to (e.g. "ICU").

Returns: bool: True if the nurse was created successfully, False if the ID already exists.

```
def CreatePatient(  
    self,  
    strPatientID,  
    strFirstName,  
    strLastName,  
    strDateOfBirth,  
    strPhoneNumber,  
    strAddress  
):
```

[•View Source](#)

Register a new patient in the system.

Args: strPatientID (str): Unique patient ID. strFirstName (str): Patient's first name. strLastName (str): Patient's last name. strDateOfBirth (str): Date of birth in YYYY-MM-DD format. strPhoneNumber (str): Contact phone number. strAddress (str): Home address.

Returns: bool: True if the patient was created successfully, False if the ID already exists.

```
def CreateLabOrder(self, strPatientID, strOrderingDoctorID, strTestName,  
    strNotes):
```

[•View Source](#)

Create a new lab order for a patient.

Args: strPatientID (str): ID of the patient the order is for. strOrderingDoctorID (str): ID of the doctor placing the order. strTestName (str): Name of the lab test (e.g. "CBC", "Urinalysis"). strNotes (str): Clinical notes or indication for the test.

Returns: bool: True if the order was created, False if the patient ID was not found.

```
def GetLabOrdersForPatient(self, strPatientID):
```

[•View Source](#)

Retrieve all lab orders associated with a specific patient.

Args: strPatientID (str): ID of the patient to filter by.

Returns: list[LabOrder]: A list of LabOrder objects belonging to the patient. Returns an empty list if none are found.

```
def GetAllLabOrders(self):
```

[•View Source](#)

Retrieve every lab order in the system.

Returns: list[LabOrder]: A shallow copy of the full lab orders list.

```
def Login(self, strRole, strIdentification, strPassword):
```

[•View Source](#)

Authenticate a user and load their permissions into the session.

Patients are not password-authenticated — any patient ID is accepted. On success, updates `currentUserCredentials` and `currentUserPermissions` with the authenticated user's data.

Args: strRole (str): Role to authenticate as. Must be "Doctor", "Nurse", or "Patient".
strIdentification (str): The user's ID / username. strPassword (str): The user's password (ignored for Patient logins).

Returns: Doctor | Nurse | Patient | bool: The authenticated role object on success, or False if authentication fails.

```
def load_all(self):
```

[•View Source](#)

Load all data from disk. Calls individual loaders for each entity type.

```
def save_all(self):
```

[•View Source](#)

Persist all in-memory data to disk. Calls individual savers for each entity type.

```
class Doctor:
```

[•View Source](#)

Represents a doctor in the medical system.

Doctors have full permissions: they can manage staff, manage patients, and perform all clinical actions including ordering lab tests.

Attributes: firstName (str): Doctor's first name. lastName (str): Doctor's last name. role (str): Always "Doctor". specialty (str): Medical specialty (e.g. "Pediatric"). permissions (dict): Full permission set granted to all doctors.

```
Doctor(  
    strIdentification,
```

```
        strPassword,  
        strFirstName,  
        strLastName,  
        strPhoneNumber,  
        strSpecialty  
    )
```

• [View Source](#)

Initialize a Doctor instance.

Args: strIdentification (str): Unique doctor ID / username. strPassword (str): Login password (stored as a private attribute). strFirstName (str): Doctor's first name. strLastName (str): Doctor's last name. strPhoneNumber (str): Contact phone number (stored as a private attribute). strSpecialty (str): Medical specialty.

firstName

lastName

role

specialty

permissions

```
def DisplayInfo(self):
```

• [View Source](#)

Print the doctor's profile information to the console.

```
def UpdateInformation(  
    self,  
    strIdentification,  
    strFirstName,  
    strLastName,  
    strPhoneNumber,  
    strSpecialty,  
    strNewPassword,  
    strCurrentPassword  
):
```

• [View Source](#)

Update the doctor's profile after verifying their current password.

Args: strIdentification (str): New doctor ID / username. strFirstName (str): New first name. strLastName (str): New last name. strPhoneNumber (str): New phone number. strSpecialty (str): New medical specialty. strNewPassword (str): New password to set. strCurrentPassword (str): Current password for verification.

Returns: tuple[bool, str]: A (success, message) tuple. - (False, error message) if the current password is incorrect. - (True, success message) if the update completed.

class Nurse:

[•View Source](#)

Represents a nurse in the medical system.

Nurses have limited permissions: they can create and view patients, view lab tests, but cannot manage staff or order lab tests.

Attributes: firstName (str): Nurse's first name. lastName (str): Nurse's last name. role (str): Always "Nurse". department (str): Department the nurse belongs to. permissions (dict): Restricted permission set granted to all nurses.

```
Nurse(
    strIdentification,
    strPassword,
    strFirstName,
    strLastName,
    strPhoneNumber,
    strDepartment
)
```

[•View Source](#)

Initialize a Nurse instance.

Args: strIdentification (str): Unique nurse ID / username. strPassword (str): Login password (stored as a private attribute). strFirstName (str): Nurse's first name. strLastName (str): Nurse's last name. strPhoneNumber (str): Contact phone number (stored as a private attribute). strDepartment (str): Department the nurse belongs to.

firstName

lastName

role

department

permissions

def DisplayInfo(self):

• View Source

Print the nurse's profile information to the console.

class Patient:

• View Source

Represents a patient in the medical system.

Patients have minimal permissions: they can only view their own profile. They cannot access staff, other patients, or clinical data.

Attributes: patient_id (str): Unique patient ID. firstName (str): Patient's first name. lastName (str): Patient's last name. dateOfBirth (str): Date of birth in YYYY-MM-DD format. role (str): Always "Patient". permissions (dict): Minimal permission set granted to all patients.

Patient(

 strPatientID,
 strFirstName,
 strLastName,
 strDateOfBirth,
 strPhoneNumber,
 strAddress

)

• View Source

Initialize a Patient instance.

Args: strPatientID (str): Unique patient ID. strFirstName (str): Patient's first name. strLastName (str): Patient's last name. strDateOfBirth (str): Date of birth in YYYY-MM-DD format. strPhoneNumber (str): Contact phone number (stored as a private attribute). strAddress (str): Home address (stored as a private attribute).

patient_id

firstName

lastName

dateOfBirth

role

permissions

def DisplayInfo(self):

• [View Source](#)

Print the patient's profile information to the console.

class LabOrder:

• [View Source](#)

Represents a single lab test order.

Each order is assigned a unique 8-character ID on creation and begins with a status of "Pending".

Attributes: `order_id` (str): Auto-generated unique 8-character uppercase order ID. `patient_id` (str): ID of the patient the order is for. `ordering_doctor_id` (str): ID of the doctor who placed the order. `test_name` (str): Name of the lab test (e.g. "CBC", "Lipid Panel"). `notes` (str): Clinical notes or indication for the test. `status` (str): Current order status. Defaults to "Pending".

LabOrder(strPatientID, strOrderingDoctorID, strTestName, strNotes)

• [View Source](#)

Initialize a LabOrder instance.

Args: `strPatientID` (str): ID of the patient the order is for. `strOrderingDoctorID` (str): ID of the doctor placing the order. `strTestName` (str): Name of the lab test. `strNotes` (str): Clinical notes or indication.

order_id

patient_id

ordering_doctor_id

test_name

notes

status

```
def Display(self):
```

[•View Source](#)

Print the lab order's details to the console.

```
def ClearConsole(intTimeout=0):
```

[•View Source](#)

Clear the terminal screen, optionally after a delay.

Args: intTimeout (int | float): Seconds to wait before clearing. Defaults to 0 (immediate).

```
def prompt(label, secret=False):
```

[•View Source](#)

Prompt the user for input, with optional hidden input for passwords.

Args: label (str): The prompt label displayed to the user. secret (bool): If True, input is hidden (suitable for passwords). Defaults to False.

Returns: str: The user's input, stripped of leading/trailing whitespace.

```
def pause():
```

[•View Source](#)

Pause execution until the user presses Enter.

```
def run_menu(title, options):
```

[•View Source](#)

Display a numbered menu and dispatch the selected action in a loop.

Continues looping until the user selects option 0 to return.

Args: title (str): Title displayed at the top of the menu. options (list[tuple[str, callable]]): A list of (label, function) pairs. Each function is called with no arguments when selected. Option 0 is always implicitly "Return" and exits the loop.

```
def ViewStaffScreen():
```

[•View Source](#)

Display all doctors and nurses currently registered in the system.

```
def CreateDoctorScreen():
```

[•View Source](#)

Prompt for new doctor details and register them in the system.

```
def CreateNurseScreen():
```

[•View Source](#)

Prompt for new nurse details and register them in the system.

```
def StaffManagementMenu(staffPerms):
```

[•View Source](#)

Display the Staff Management submenu with options filtered by the user's permissions.

Args: staffPerms (dict): The "StaffManagement" slice of the current user's permissions dict. Expected keys: "createsDoctors" (bool), "createsNurses" (bool).

```
def CreatePatientScreen():
```

[•View Source](#)

Prompt for new patient details and register them in the system.

```
def ViewAllPatientsScreen():
```

[•View Source](#)

Display all patients currently registered in the system.

```
def ViewPatientByIDScreen():
```

[•View Source](#)

Prompt for a patient ID and display that patient's information.

```
def PatientManagementMenu(patientPerms):
```

[•View Source](#)

Display the Patient Management submenu with options filtered by the user's permissions.

Args: patientPerms (dict): The "PatientManagement" slice of the current user's permissions dict. Expected keys: "createsPatients" (bool), "readsPatientDetails" (bool).

```
def OrderLabTestScreen():
```

[•View Source](#)

Prompt the logged-in doctor to order a lab test for a selected patient.

Displays all available patients before prompting for input. Automatically uses the currently logged-in doctor's ID as the ordering doctor.

```
def ViewAllLabOrdersScreen():
```

[•View Source](#)

Display all lab orders currently in the system.

```
def ViewLabOrdersByPatientScreen():
```

[•View Source](#)

Prompt for a patient ID and display all lab orders associated with that patient.

```
def ClinicalActionsMenu(clinicalPerms):
```

[•View Source](#)

Display the Clinical Actions submenu with options filtered by the user's permissions.

Args: clinicalPerms (dict): The "ClinicalActions" slice of the current user's permissions dict.
Expected keys: "ordersLabTests" (bool), "viewsLabTests" (bool).

```
def CredentialsMenu(role):
```

[•View Source](#)

Prompt for credentials and authenticate the user, retrying on failure.

Args: role (str): The role to authenticate as ("Doctor", "Nurse", or "Patient").

Returns: Doctor | Nurse | Patient: The authenticated role object on success.

```
def LoginMenu():
```

[•View Source](#)

Prompt the user to select a role and log in.

Loops until a valid role is entered, then delegates to `CredentialsMenu`.

Returns: Doctor | Nurse | Patient: The authenticated role object.

```
def userMenu(currentUserRoleObject):
```

[•View Source](#)

Display the main menu for the logged-in user, showing only permitted options.

Loops until the user selects exit (option 0), at which point all data is saved.

Args: currentUserRoleObject (Doctor | Nurse | Patient): The authenticated user object returned by `LoginMenu`. Used to call `DisplayInfo` for the user's own profile.